

Case Study Report
on
Automating Docker Image Build and Push to
Docker Hub using Jenkins
Bachelor of Technology
in
Computer Science Engineering (Artificial Intelligence & Machine Learning)

By

Student Name

Roll No

Gunna Pavan

A23126552265

Nalli Bobby Raj

A23126552267

Nazeer Ahmad

A23126552269

Rapaka Dinesh

A23126552271

Mondi Naga Durga Ramesh

A23126552272



DEPARTMENT OF COMPUTER SCIENCE ENGINEERING (AI & ML)
ANIL NEERUKONDA INSTITUTE OF TECHNOLOGY & SCIENCES

Sangivalasa, Bheemunipatnam Mandal,

Visakhapatnam Dist, AP, India

2026



CERTIFICATE

The case study report entitled “**Automating Docker Image Build and Push to Docker Hub using Jenkins**” submitted by Gunna Pavan, Nalli Bobby Raj, Nazeer Ahmad, Rapaka Dinesh, Mondy Naga Durga Ramesh III/IV B.Tech, II-Semester bearing Regd. No. A23126552265, A23126552267, A23126552269, A23126552271, A23126552272 Dept. of CSE(AI&ML) is found to be satisfactory.

Mr. Santosh Kumar P
Associate Professor
Dept. of CSE(AI&ML)

ACKNOWLEDGEMENT

This case study reflects our academic learning and professional development journey. The case study provided an opportunity to apply classroom knowledge in a real-world cloud and containerization environment.

We express sincere gratitude to Dr. K. Selvani Deepthi, Head of the Department of Computer Science and Engineering (AI & ML) at ANITS, and our guide Mr. Santosh Kumar P, Associate Professor, for invaluable guidance and encouragement throughout the case study. We also acknowledge the faculty members for their unwavering support.

Regd No.	A23126552265
	A23126552267
	A23126552269
	A23126552271
	A23126552272

CONTENTS

Sl. No.	Title	Page No.
1	Abstract	
2	Introduction to DevOps	
3	Objectives of the Project	
4	Tools and Technologies Used	
5	System Architecture	
6	Prerequisites	
7	Step-by-Step Implementation	
8	Project Files Explanation	
9	Output Explanation	
10	Advantages	
11	Limitations	
12	Conclusion	
13	Future Enhancements	
14	References	

1. Abstract

This project demonstrates the design and implementation of a Continuous Integration (CI) pipeline using Jenkins, Docker, and Docker Hub on an AWS EC2 instance. The objective is to automate the process of building a Docker image from application source code stored on GitHub and pushing it to Docker Hub — entirely without manual intervention.

The application is a simple HTML web page served using Nginx, a popular lightweight web server. Jenkins monitors the GitHub repository, builds a Docker image on every code update, logs in to Docker Hub securely, and pushes the final image. This project illustrates the core DevOps principle of automation and serves as a practical foundation for understanding CI pipelines in real-world software development.

2. Introduction to DevOps

DevOps is a cultural and technical movement that bridges the gap between software development (Dev) and IT operations (Ops). It promotes collaboration, automation, and continuous delivery to improve the speed and quality of software releases. Modern development teams rely heavily on DevOps practices to reduce errors, speed up deployments, and ensure reliable software delivery.

2.1 Key Principles of DevOps

- Continuous Integration (CI): Frequently merging code changes and automatically testing them to detect issues early.
- Continuous Delivery (CD): Automatically delivering tested code to staging or production environments.
- Infrastructure as Code: Managing servers and environments using version-controlled scripts.
- Monitoring and Feedback: Continuously observing systems to improve reliability and performance.

2.2 Importance of CI/CD Pipelines

A CI/CD pipeline automates the steps from writing code to deploying it. This reduces human errors, speeds up delivery, and ensures that every build is tested and traceable. Jenkins is one of the most widely used open-source CI tools that enables the creation of such automated pipelines.

Docker allows developers to package applications with all their dependencies into portable containers. When combined with Jenkins, Docker enables consistent and reproducible builds across any environment — from a developer's laptop to a cloud server.

3. Objectives of the Project

The main objectives of this project are:

1. To set up Jenkins on an AWS EC2 instance for automating CI tasks.
2. To install and configure Docker on the same server.
3. To create a Jenkins pipeline that automatically builds a Docker image from code.
4. To securely push the built Docker image to Docker Hub using Jenkins credentials.
5. To understand the complete CI flow from source code commit to image deployment.
6. To use a Jenkinsfile for pipeline-as-code configuration stored in GitHub.
7. To serve a simple HTML web page using an Nginx-based Docker container.

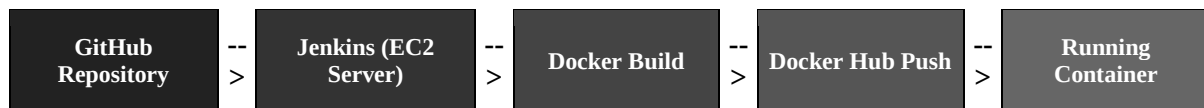
4. Tools and Technologies Used

Tool / Technology	Purpose
AWS EC2 (Ubuntu Linux)	Cloud server used to host Jenkins and Docker. Ubuntu 22.04 LTS was used.
Jenkins	Open-source automation server used to create and run the CI pipeline.
Docker	Containerization platform to build and run application containers.
Docker Hub	Cloud registry used to store and distribute Docker images (ramesh573/app).
GitHub	Source code management platform storing code, Dockerfile, and Jenkinsfile.
Nginx	Lightweight web server used inside the Docker container to serve the HTML page.

5. System Architecture

The architecture follows a simple and linear CI pipeline flow. The developer pushes code to GitHub, Jenkins detects the change, builds a Docker image, and pushes it to Docker Hub. The image can then be pulled and run as a container on any machine.

5.1 Architecture Flow



5.2 Flow Explanation

- **GitHub Repository:** The developer pushes application code (index.html, Dockerfile, Jenkinsfile) to the GitHub repository xramesh57/docker-jenkins-app.
- **Jenkins (on AWS EC2):** Jenkins clones the repository and executes the pipeline stages defined in the Jenkinsfile.
- **Docker Build:** Jenkins runs docker build to create a Docker image named ramesh573/app.
- **Docker Hub Push:** Jenkins logs in to Docker Hub using stored credentials and pushes the image.
- **Running Container:** The Docker image is pulled from Docker Hub and run as a container, serving the HTML page via Nginx on port 80.

6. Prerequisites

Before starting the implementation, ensure the following are in place:

6.1 Accounts and Software

- An AWS account with an EC2 instance (Ubuntu 22.04 LTS, t2.medium or higher).
- Security Group configured to allow inbound traffic on Port 22 (SSH), Port 8080 (Jenkins), and Port 80 (HTTP).
- A GitHub account with a repository containing the project files (Dockerfile, Jenkinsfile, index.html).
- A Docker Hub account with a repository to store the Docker image.
- Basic knowledge of Linux commands and SSH access.

6.2 EC2 Instance Configuration

Parameter	Recommended Value
Instance Type	t2.medium (Jenkins + Docker)
OS / AMI	Ubuntu Server 22.04 LTS
Storage	20 GB (minimum)
Key Pair	Required for SSH access
Security Ports	22 (SSH), 8080 (Jenkins), 80 (Web)

7. Step-by-Step Implementation

Follow the steps below in order. All commands are to be run on the AWS EC2 Ubuntu terminal after SSH login.

Step 1: Connect to EC2 Instance

Open a terminal and connect to the EC2 instance using SSH:

```
ssh -i your-key.pem ubuntu@<your-ec2-public-ip>
```

Step 2: Update the System

```
sudo apt update -y
sudo apt upgrade -y
```

Step 3: Install Docker

Install Docker and add users to the Docker group:

```
sudo apt install docker.io -y
sudo systemctl start docker
sudo systemctl enable docker
```

```
sudo usermod -aG docker jenkins
sudo usermod -aG docker ubuntu
docker --version
```

Step 4: Install Java (Required for Jenkins)

```
sudo apt install openjdk-17-jdk -y
java -version
```

Step 5: Install Jenkins

```
curl -fsSL https://pkg.jenkins.io/debian-stable/jenkins.io-2023.key | sudo tee \
  /usr/share/keyrings/jenkins-keyring.asc > /dev/null

echo deb [signed-by=/usr/share/keyrings/jenkins-keyring.asc] \
  https://pkg.jenkins.io/debian-stable binary/ | sudo tee \
  /etc/apt/sources.list.d/jenkins.list > /dev/null

sudo apt update
sudo apt install jenkins -y
```

Step 6: Start Jenkins

```
sudo systemctl start jenkins
sudo systemctl enable jenkins
sudo systemctl status jenkins
```

Step 7: Unlock Jenkins

Open a browser and navigate to: <http://<your-ec2-ip>:8080>

Retrieve the initial admin password:

```
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

Paste the password in the Jenkins setup wizard and install suggested plugins.

Step 8: Install Required Jenkins Plugins

Go to: Dashboard > Manage Jenkins > Plugins > Available Plugins. Search and install:

- Docker Pipeline
- Git Plugin
- Credentials Binding Plugin

Step 9: Add Docker Hub Credentials to Jenkins

Go to: Dashboard > Manage Jenkins > Credentials > System > Global Credentials > Add Credentials

Field	Value
Kind	Username with password
Username	Your Docker Hub username (ramesh573)
Password	Your Docker Hub password or access token
ID	dockerhub-creds
Description	Docker Hub Credentials

Step 10: Create a Jenkins Pipeline Job

Go to: Dashboard > New Item. Enter job name: docker-jenkins-pipeline, select Pipeline, and click OK.

1. Under Pipeline Definition, select: Pipeline script from SCM.
2. Set SCM to Git and paste your GitHub repository URL.
3. Set branch to: */main and Script Path to: Jenkinsfile.
4. Click Save and then Build Now to trigger the pipeline.

Step 11: Key Docker Commands Reference

The following commands are executed by the Jenkins pipeline stages:

Docker Build Command:

```
docker build -t ramesh573/app .
```

Docker Login Command:

```
echo $PASS | docker login -u $USER --password-stdin
```

Docker Push Command:

```
docker push ramesh573/app
```

Run Container After Pulling from Docker Hub:

```
docker run -d -p 80:80 ramesh573/app
```

8. Project Files Explanation

This project uses three key files. Each file is explained below with its full content and purpose.

8.1 Dockerfile

The Dockerfile defines the steps to build the Docker image. It uses the official Nginx Alpine image as the base and copies all project files into the Nginx web root directory.

```
FROM nginx:alpine
COPY . /usr/share/nginx/html
```

Instruction	Meaning
FROM nginx:alpine	Use Nginx on Alpine Linux as the base image (small and efficient).
COPY . /usr/share/nginx/html	Copy all project files into Nginx's default web folder.

8.2 Jenkinsfile

The Jenkinsfile defines the complete CI pipeline using Declarative Pipeline syntax. It contains 4 stages: Checkout Code, Build Docker Image, Login to DockerHub, and Push Image. Docker Hub credentials are accessed securely using Jenkins' built-in credentials binding, so passwords are never stored in plain text.

```
pipeline {
  agent any

  environment {
    IMAGE_NAME = "ramesh573/app"
  }

  stages {

    stage('Build Docker Image') {
      steps {
        sh 'docker build --no-cache -t $IMAGE_NAME .'
      }
    }

    stage('Login to DockerHub') {
      steps {
        withCredentials([usernamePassword(credentialsId: 'dockerhub-creds',
usernameVariable: 'USER', passwordVariable: 'PASS')]) {
          sh 'echo $PASS | docker login -u $USER --password-stdin'
        }
      }
    }
  }
}
```

```

    }

    stage('Push Image') {
        steps {
            sh 'docker push $IMAGE_NAME'
        }
    }

    stage('Deploy Container') {
        steps {

```

Stage	Description
Checkout Code	Clones the application code from the GitHub repository (main branch).
Build Docker Image	Runs docker build to create a Docker image tagged as ramesh573/app.
Login to DockerHub	Securely logs into Docker Hub using credentials stored in Jenkins.
Push Image	Pushes the built Docker image to Docker Hub for public access.

8.3 index.html

The index.html file is the web application served by Nginx inside the container. It displays a simple message confirming the DevOps pipeline is working:

```

<!DOCTYPE html>
<html>
  <head><title>DevOps Project</title></head>
  <body>
    <h1>DevOps Project Working &#128640;</h1>
    <h2>Version 1</h2>
  </body>
</html>

```

This is the exact content that was committed to GitHub, built into the Docker image, and rendered in the browser output shown in the project screenshots.

9. Output Explanation

After the Jenkins pipeline executes successfully, several visible outputs confirm that the CI process worked correctly. The outputs are presented below with actual screenshots from the project.

9.1 GitHub Repository

The GitHub repository `xramesh57/docker-jenkins-app` contains three project files: `Dockerfile`, `Jenkinsfile`, and `index.html`. The repository has 27 commits, reflecting multiple iterative builds triggered by code changes.

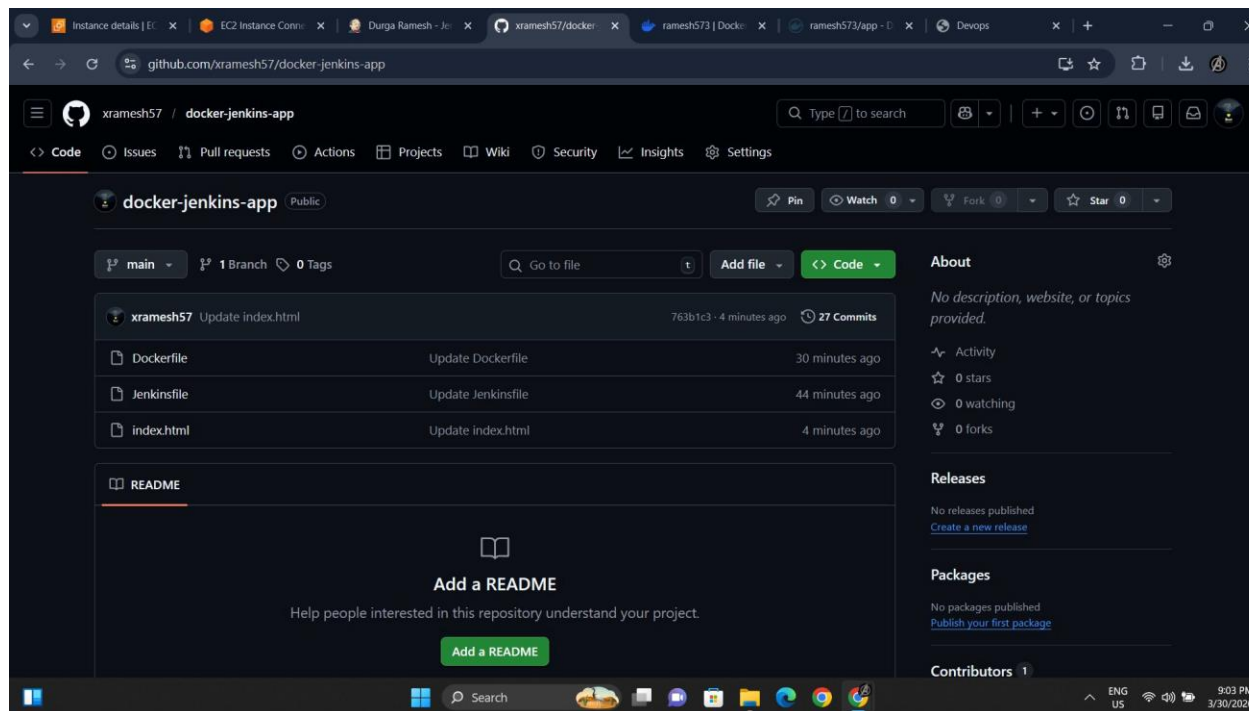


Figure 1: GitHub repository showing `Dockerfile`, `Jenkinsfile`, and `index.html`

9.2 Docker Hub — Image Tags (First Build)

After the first successful pipeline run, the Docker image `ramesh573/app` was pushed to Docker Hub with the 'latest' tag. The image has digest `e031e03bfc14`, runs on `linux/amd64` architecture, and has a compressed size of 24.83 MB. The repository has recorded 76 pulls.

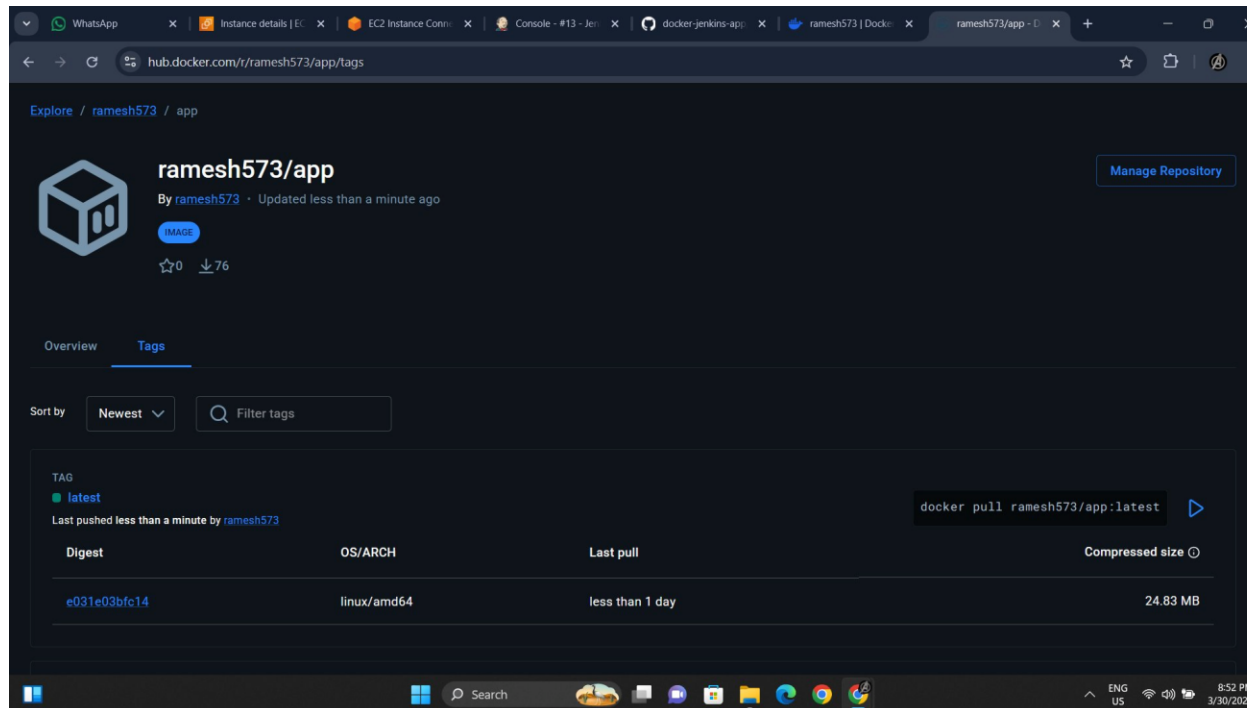


Figure 2: Docker Hub showing ramesh573/app:latest tag with digest e031e03bfc14 (24.83 MB)

9.3 Web Output — Version 1

After running the Docker container from the pushed image on the EC2 server, the browser accessed <http://15.134.219.182> and displayed the HTML web page successfully. The page confirms the DevOps pipeline is fully operational.

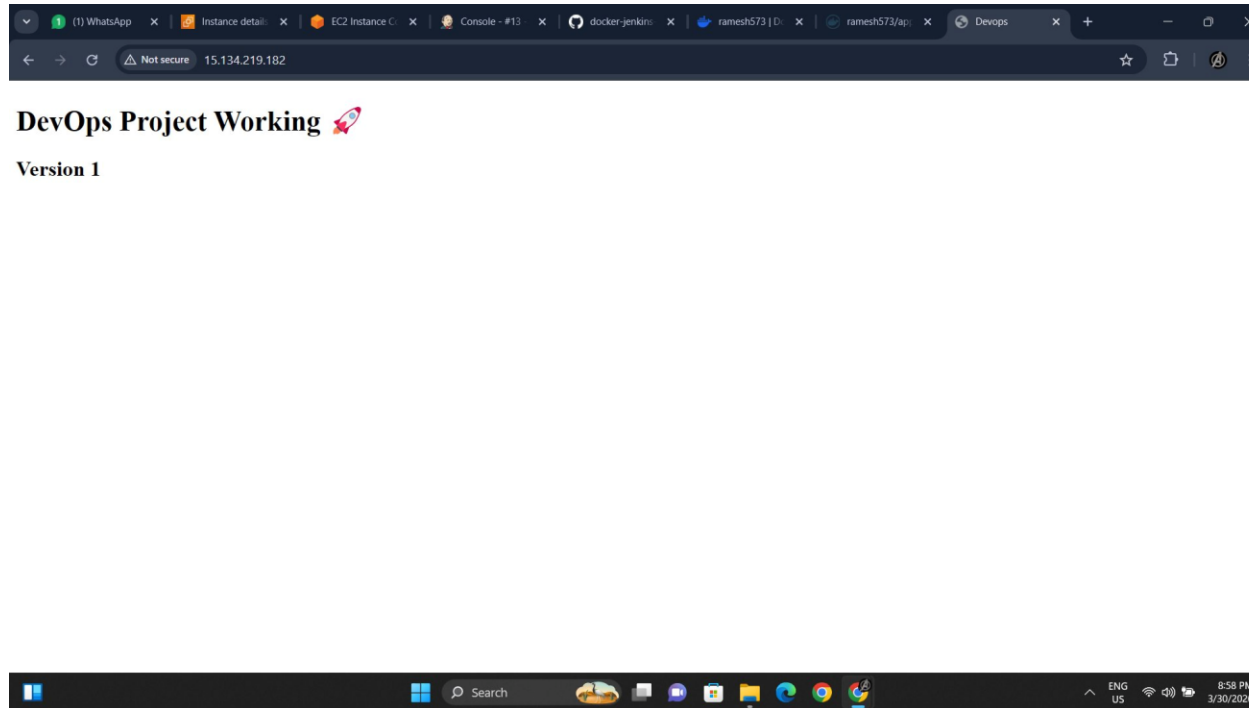


Figure 3: Browser output showing 'DevOps Project Working' Version 1 at 15.134.219.182

9.4 Docker Hub — Multiple Image Tags (After Re-runs)

After additional pipeline runs triggered by code changes (index.html updated to Version 2), Docker Hub shows multiple tags. The latest and build 14 tags share the same digest 65a0bca5c2f7, while tag 13 has digest e031e03bfc14. This demonstrates that the pipeline creates a new image for every successful Jenkins build.

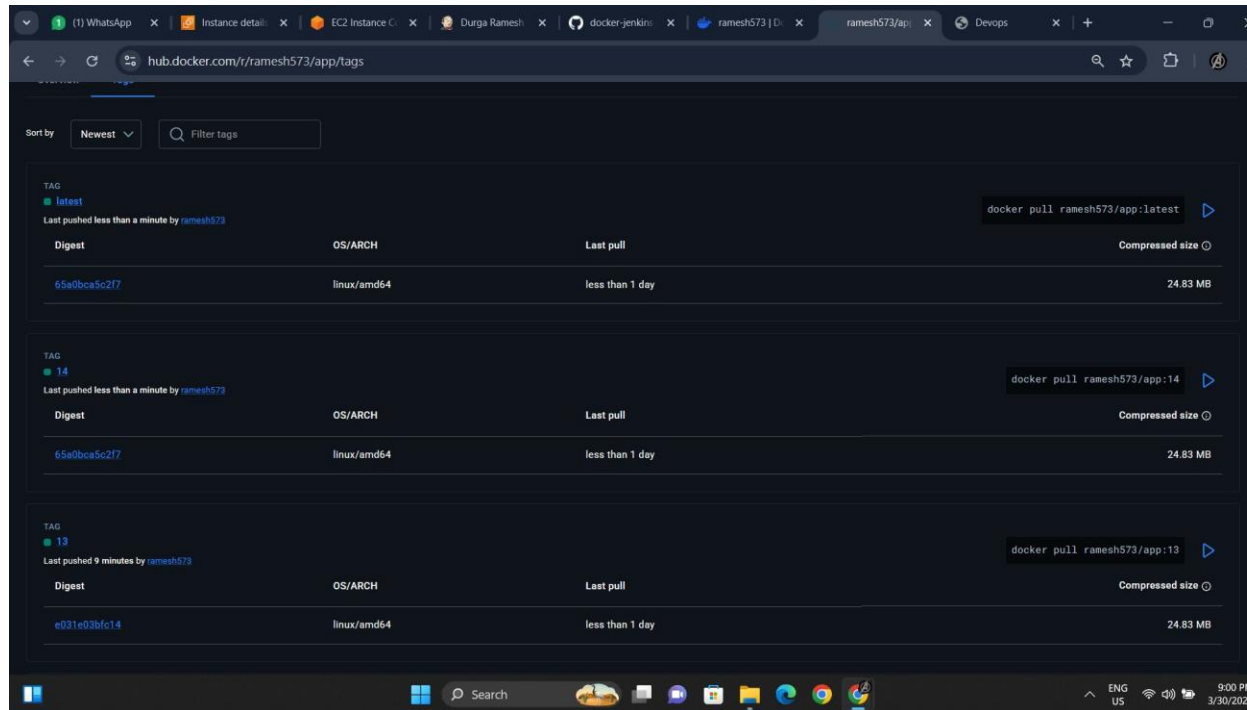


Figure 4: Docker Hub showing multiple tags — latest, 14, and 13 with their respective digests

9.5 Web Output — Version 2

After updating index.html on GitHub to display Version 2, the Jenkins pipeline was automatically triggered. A new Docker image was built and pushed. Running the updated container displayed Version 2 in the browser, confirming end-to-end CI pipeline automation is working.

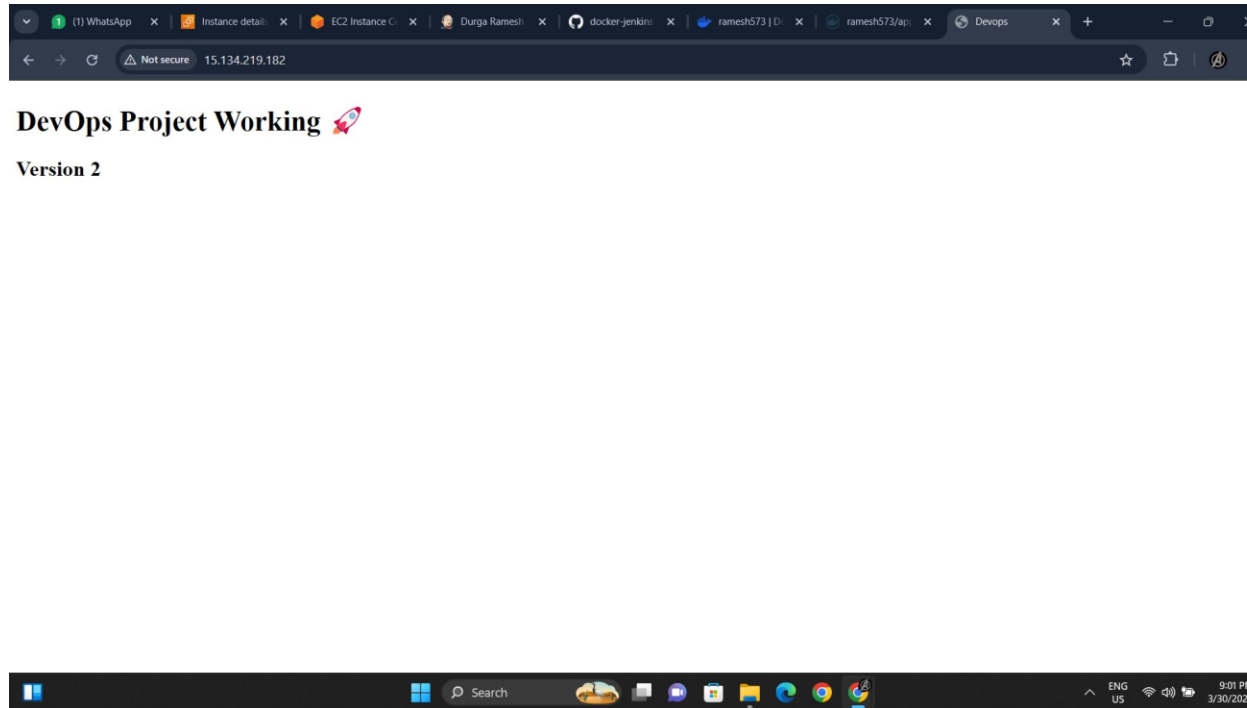


Figure 5: Browser output showing 'DevOps Project Working' Version 2 — updated via CI pipeline

9.6 Result Summary

Test Case	Expected Result	Status
Jenkins pipeline executes all 4 stages	Success	PASS
Docker image built from Dockerfile	Built	PASS
Image pushed to Docker Hub as ramesh573/app	Pushed	PASS
Web page accessible at http://15.134.219.182	Accessible	PASS
Version 2 update triggers new build automatically	Deployed	PASS

10. Advantages of the Project

- **Full Automation:** The entire process from code commit to Docker image push is automated. No manual steps are required after the initial setup.
- **Consistency:** Docker ensures the application behaves identically in development, testing, and production environments.
- **Portability:** The Docker image can be pulled and run on any machine or cloud platform supporting Docker.
- **Speed:** Jenkins executes the pipeline in minutes, drastically reducing delivery time compared to manual processes.
- **Security:** Docker Hub credentials are stored securely in Jenkins using the Credentials plugin. Passwords are never exposed in the Jenkinsfile.
- **Reusability:** The same pipeline structure can be reused for different projects with minor modifications to the Jenkinsfile.
- **Version Control:** Both the application code and the pipeline script are version-controlled in GitHub, supporting traceability.
- **Scalability:** The pipeline can be extended to support multiple environments, testing stages, and deployment strategies.

11. Limitations

- **No Automated Testing:** The pipeline does not include unit or integration tests. A broken application could still be pushed to Docker Hub.
- **No Continuous Deployment:** The pipeline only pushes the image to Docker Hub. It does not automatically deploy the container to any server.
- **Single Tag:** The pipeline pushes using the 'latest' tag by default, making version rollback difficult.
- **No Notifications:** The pipeline does not send alerts via email or Slack when a build fails.
- **Resource Constraints:** Running Jenkins and Docker on a small EC2 instance may cause slowness during parallel builds.
- **No Load Balancing:** A single container is run manually. There is no auto-scaling or load balancing in this project.
- **HTTP Only:** The application runs over HTTP without SSL/TLS, making it unsuitable for production without additional configuration.

12. Conclusion

This project successfully demonstrated the implementation of a Continuous Integration pipeline using Jenkins and Docker on AWS EC2. The pipeline automates the complete workflow: fetching code from GitHub, building a Docker image, and pushing it to Docker Hub — without any manual intervention.

The use of a Jenkinsfile makes the pipeline version-controlled and easy to modify. Docker ensures that the application is packaged consistently and can run anywhere. Docker Hub acts as a centralized registry from which the image can be downloaded and deployed on demand.

Real project outputs — including Docker Hub image tags with digest hashes, multiple versioned builds, and browser screenshots of the running web application — confirm that the pipeline works end-to-end. This project provides hands-on experience with Jenkins, Docker, and GitHub, which are widely used in modern software development and DevOps teams.

13. Future Enhancements

13.1 Continuous Deployment (CD)

An additional pipeline stage can automatically deploy the container to a server or cloud platform (such as AWS ECS or EC2) after the image is pushed. This would make the pipeline a full CI/CD workflow.

13.2 Versioned Image Tags

Instead of always using the 'latest' tag, the pipeline can be modified to tag images using the Jenkins build number or Git commit hash (e.g., ramesh573/app:14). This supports version tracking and easy rollback.

```
sh 'docker build -t $IMAGE_NAME:$BUILD_NUMBER .'
```

13.3 Automated Testing Stage

A testing stage can be added before the build stage to run unit tests. The pipeline will only proceed to build and push if all tests pass, ensuring code quality.

13.4 Kubernetes Orchestration

The Docker image pushed to Docker Hub can be deployed to a Kubernetes cluster (e.g., Amazon EKS or minikube). Kubernetes provides auto-scaling, self-healing, rolling updates, and load balancing for containerized applications — making this a production-grade deployment solution.

13.5 Build Notifications

Jenkins can be configured to send build status notifications via email or Slack when a build succeeds or fails. This helps teams respond to issues quickly without manually checking Jenkins.

13.6 Nginx Reverse Proxy and HTTPS

An Nginx reverse proxy can be configured in front of the container to add SSL/TLS termination, enabling HTTPS access and making the application production-ready.

References

1. Jenkins Official Documentation. Available: <https://www.jenkins.io/doc/>
2. Docker Official Documentation. Available: <https://docs.docker.com/>
3. Docker Hub — ramesh573/app. Available: <https://hub.docker.com/r/ramesh573/app>
4. GitHub Repository — xramesh57/docker-jenkins-app. Available: <https://github.com/xramesh57/docker-jenkins-app>
5. AWS EC2 Documentation. Available: <https://docs.aws.amazon.com/ec2/>
6. Nginx Official Documentation. Available: <https://nginx.org/en/docs/>